



Game Client Programming Portfolio

클라이언트 프로그래밍 포트폴리오

이다건

010 - 5954 - 0403

leedageon764@gmail.com

○ 목차

❖ 이력

❖ EisenValor 졸업작품

- Inverse Kinematics
- UI Architecture
- Finite State Machine
- Model & Animation Fix
- Socket System & Root Motion << 어떻게 넣을지 고민중
- 최적화 사항

❖ 3D게임프로그래밍 프로젝트

- Motion Blur
- Water Rending
- GPU Instancing Billboard Rendering
- Shadow Mapping
- Sparkle

❖ 기타 프로젝트

- **TinyMMORPG**
IOCP Game Server
- **Shader Effects**
OpenGL GLSL
- **Zombie Shooting Game**
Unreal Engine Blueprint
- **NightMareAdventure**
Python
- **Chicken Run**
Window API

❖ AI 활용

- 프로그래밍 명세서

○ 이력

○ Inverse Kinematics

개요

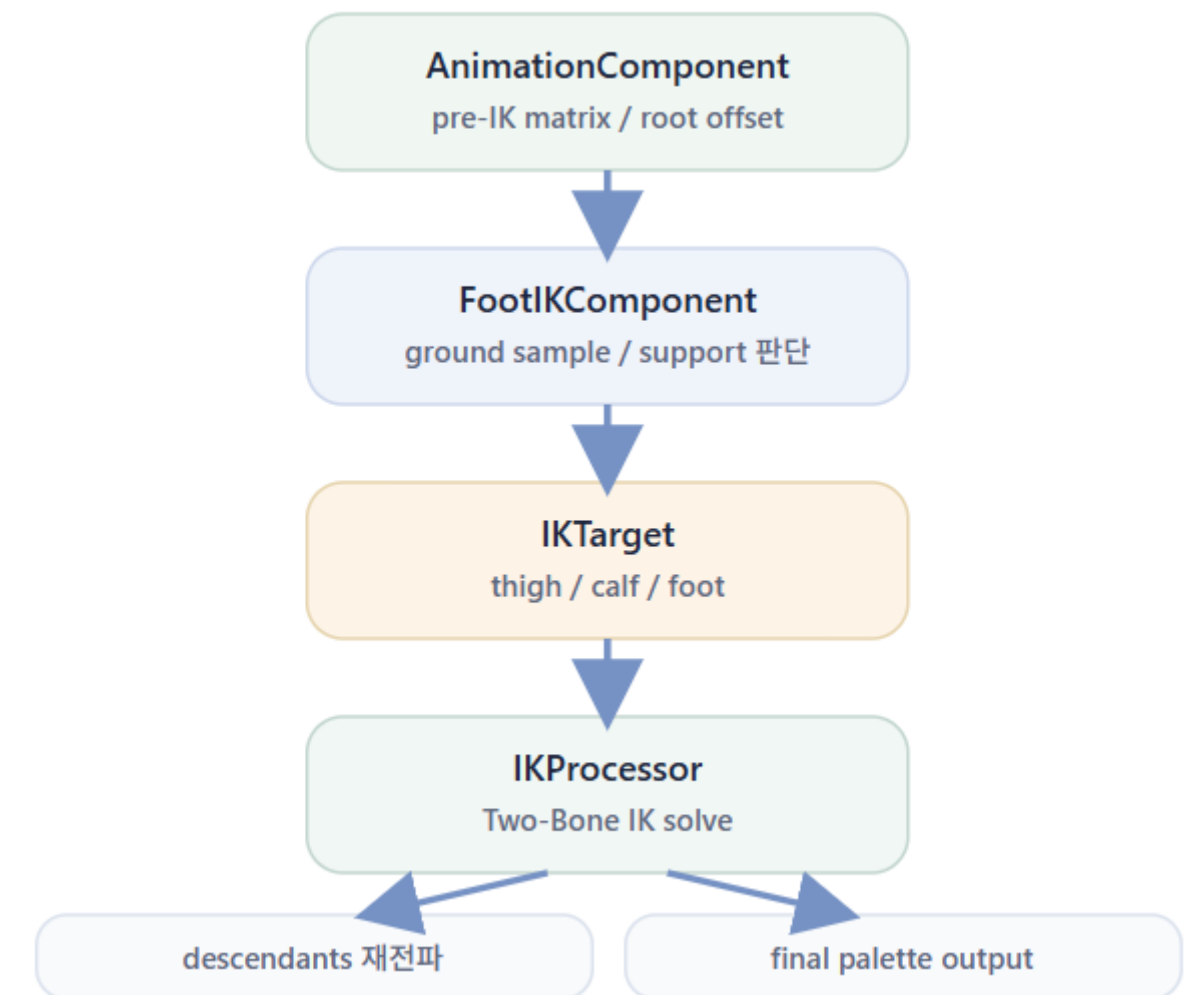
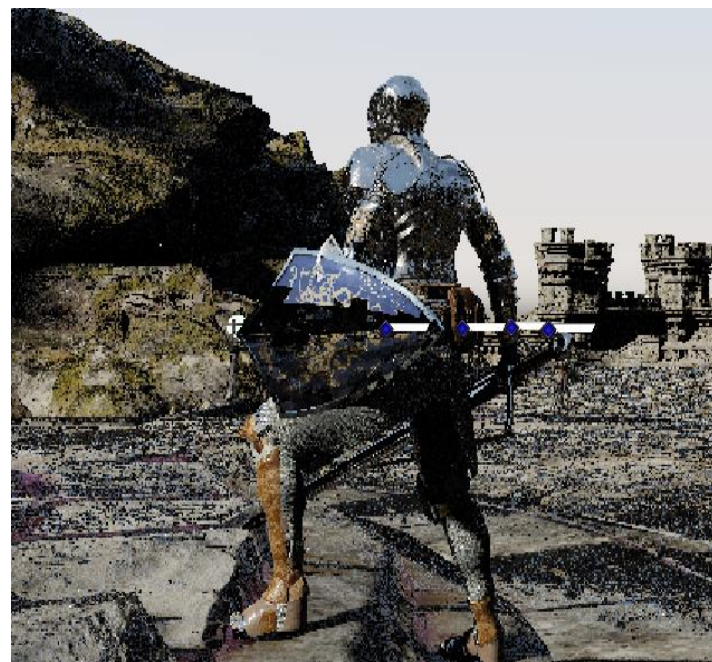
무엇을 구현했나

전투용 IDLE, 이동, 회피 상태 위에 Foot IK를 적용해 발이 지형에 맞게 자연스럽게 보이도록 한다. 무릎 방향, pelvis 높이, 애니메이션 보간까지 함께 다룬다.

왜 필요한가

원본 애니메이션만 사용하면 계단, 경사면, 낮은 지형에서 발이 뜨거나 무릎이 비정상적으로 꺾인다. 따라서 런타임 지면 정보를 받아 자세를 보정한다.

AnimationComponent가 먼저 원본 본 행렬을 계산하고, 이를 pre-IK global matrix로 별도 보존한다. 이후 FootIKComponent가 각 발 본의 world 위치를 기준으로 아래 방향 ray를 쏘아 지면 hit를 탐색하고, 그 결과를 이용해 발 목표 위치와 pelvis offset을 계산한다. 이렇게 만든 목표값은 IKTarget에 thigh / calf / foot 체인 정보와 함께 정리되고, IKProcessor가 Two-Bone IK를 이용해 허벅지-종아리-발 구조를 보정한다. IK 적용이 끝나면 수정된 본의 자식 계층을 다시 전파하고, Final Palette를 갱신해 스키닝에 사용할 최종 본 팔레트를 만든다.



○ Inverse Kinematics

❖ 문제와 해결



지면 샘플 결과를 즉시 반영하면서 발 위치와 가중치가 프레임마다 급하게 변해 계단과 경사면에서 순간적으로 무릎이 반대로 꺾이는 문제가 발생했다.



원본 애니메이션의 pre-IK 본 행렬을 별도로 저장하고, 허벅지-종아리 방향을 pole vector 힌트로 사용해 무릎의 굽힘 방향을 안정화했다.

SmoothApproach를 적용해 발 weight와 pelvis offset이 목표 값으로 한 번에 점프하지 않고 시간에 따라 점진적으로 수렴하도록 만들어 IK 결과의 시각적 흔들림을 완화했다.

Finite State Machine

개요

무엇을 구현했나

입력 코드가 바로 상태를 바꾸지 않고, 모든 전이를
RequestState() 로 모아 StatePolicy 가 판정한 뒤
ChangeState() 를 수행하는 정책 기반 FSM을 구현했다.

왜 필요했나

전투 액션 게임은 입력 반응성, 공격 모션 일관성,
서버 보정 수용을 동시에 만족해야 한다. 상태 전이 규칙이 분산되면
이 세 조건이 쉽게 충돌한다.

1

요청 기반 진입점으로 통합

이동, 공격, 구르기, 스텝, 서버 보정은 먼저 RequestState로 들어간다.
이 단계에서 플레이어와 병사 객체를 분리하고, 병사는 ForcedServerCorrection만 허용한다.

2

정책 객체가 전이 가능 여부를 판정

StatePolicy에서 현재 상태와 요청 종류를 보고 전이를 허용, 거절, 강제 보정 중 하나로 결정한다.
공격 중에는 이동과 서버 보정을 막아 전투 흐름을 보호한다.

3

상태 실행과 애니메이션을 분리

StatePool에서 상태 객체를 보관하고, StateImplementation이 실제 동작을 담당한다.
StateImplementation에서는 애니메이션 키를 계산하고 타이머를 누적해 다음 단계로 넘긴다.

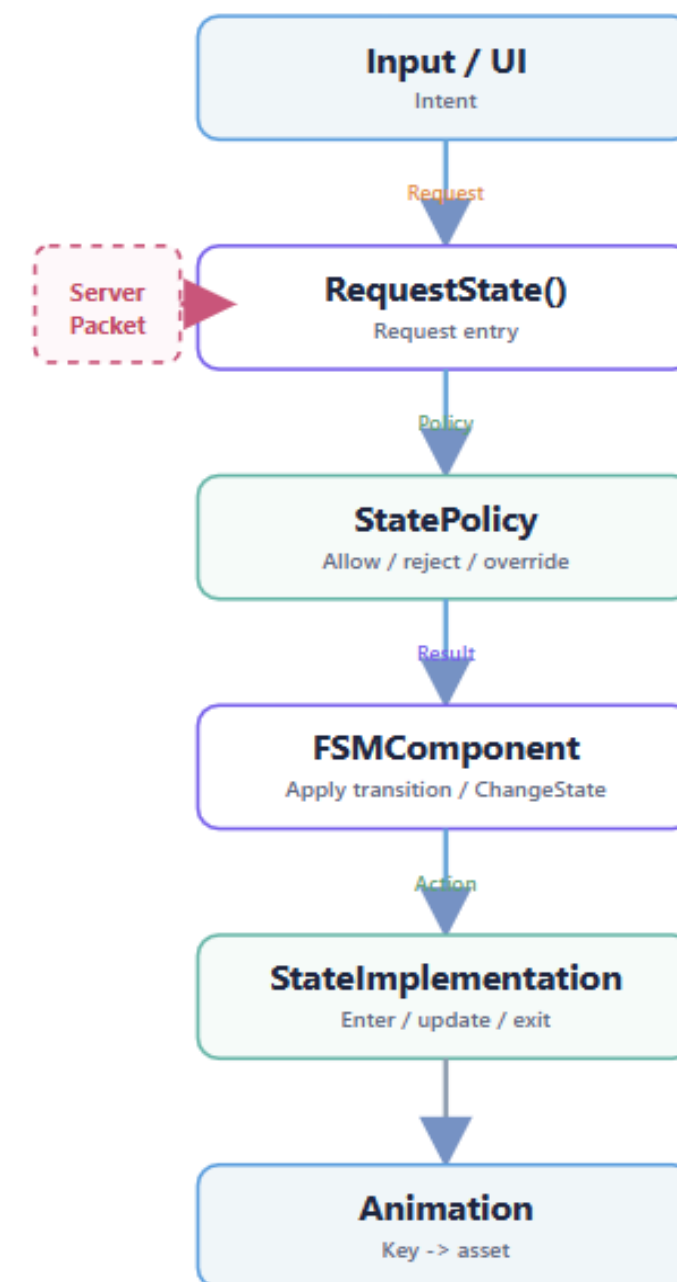
4

공격은 3단계 시간 모델로 분해

AttackTiming 구조체가 공격별 PreDelay / Attack / PostDelay를 분리한다.
선딜레이, 판정 구간, 후딜레이를 제어해 조작감과 네트워크 보정 충돌을 줄인다.

5

Animation 단계에서 최종 키 재생 및 서버 상태와 수렴



○ Finite State Machine

❖ 문제와 해결



이동, 공격, 피격, 서버 보정이 각 코드에서 직접 상태를 바꾸면 예외 처리가 분산되고 충돌 원인을 찾기 어렵다.



모든 전이를 RequestState() 하나로 모으고, 실제 판단은 StatePolicy에 집중시켰다.



공격 선딜과 후딜 중 다른 요청이 섞이면 모션이 끊기거나 공격 판정 흐름이 깨진다.



PRE_DELAY, ATTACK, POST_DELAY를 공격 시퀀스로 묶고, StatePolicy가 이 구간에서 이동과 서버 보정을 거절하게 했다



공격별로 선딜레이와 후딜레이 시간이 다르다.
단일 ATTACK 상태만 사용하면 공격 종류별 시간 구간을 독립적으로 제어할 수 없다.



AttackTiming 구조체로 공격 유형마다 서로 다른 시간 파라미터를 적용했다.

Finite State Machine

❖ 논문 작성 및 포스터 제작

2026 한국게임학회 춘계 학술발표대회 포스터 발표

전투 액션 게임의 상태 전이 제어와 서버 동기화를 위한 정책 기반 FSM 구조

이다건[○]

한국공학대학교 게임공학과
2023112023@tukorea.ac.kr

Design of a Policy-Based FSM Structure for State Transition Control and Server Synchronization in Combat Action Games

Da-Geon Lee[○]

Department of Game&Multimedia Engineering, Tech University of Korea

요 약

전투 액션 게임에서는 입력 반응성, 애니메이션 일관성, 서버 상태 보정이 동시에 요구된다. 그러나 기존 FSM 구조에서는 입력 처리, 상태 전이, 애니메이션 재생, 서버 보정의 책임이 여러 계층에 분산되기 쉽다.

본 논문은 전투 액션 게임 클라이언트에 적용된 정책 기반 FSM 구조를 설명한다. 해당 구조는 입력을 FSM 기반 상태 전이 요청으로 변환하고, 전이 정책을 통해 상태 실행, 애니메이션 재생, 서버 보정을 일관된 흐름으로 관리한다. 또한 공격 동작을 선딜레이, 실행, 후딜레이 단계로 분리하여 상태 전이와 공격 시퀀스를 독립적으로 제어한다. 이를 통해 복잡한 전투 로직과 로컬 예측, 서버 권한 상태 간 충돌을 안정적으로 처리할 수 있다.

1. 서 론

전투 액션 게임의 캐릭터 동작은 이동, 공격, 피격, 사망과 같은 다양한 상태와 예외 전이를 포함한다. 네트워크 환경에서는 여기에 서버 상태 동기화와 클라이언트 로컬 예측까지 추가된다. 특히 입력 반응성과 애니메이션 일관성을 동시에 유지하기 위해서는 상태 전이와 서버 보정을 안정적으로 관

리할 필요가 있다.

입력 계층이 직접 'ChangeState()'를 호출하는 구조에서는 입력 처리, 애니메이션 종료 조건, 서버 상태 반영 규칙이 서로 다른 코드 위치에 분산되며, 전투 중 예외 전이와 네트워크 보정 처리의 일관성을 유지하기 어렵다. 또한 클라이언트가 공격 애니메이션을 예측 재생하는 상황에서 늦게 도착한 서버 상태가 이를 즉시 덮어쓰면 입력 피드

전투 액션 게임의 상태 전이 제어와 서버 동기화를 위한 정책 기반 FSM 구조

Design of a Policy-Based FSM Structure for State Transition Control and Server Synchronization in Combat Action Games

이다건

Da-geon Lee

한국공학대학교 Tech University of Korea, Dept. of Game&Multimedia Engineering

2023112023@tukorea.ac.kr

ABSTRACT

전투 액션 게임에서는 입력 반응성, 애니메이션 일관성, 서버 상태 보정 이 동시에 요구된다. 기존 FSM 구조에서는 입력 처리, 상태 전이, 애니메이션 재생, 서버 보정의 책임이 여러 계층에 분산되기 쉽다. 본 포스터는 입력을 상태 전이 요청으로 변환하고, StatePolicy를 통해 허용·거절·즉시 전이를 관리하는 정책 기반 FSM 구조를 정리한다. 공격 동작은 Pre-delay, Attack, Post-delay 단계로 분리하며, 서버 패킷은 ForcedServerCorrection 요청으로 수립시켜 로컬 예측과 서버 권한 상태의 충돌을 완화한다.

연구 목적

RESEARCH OBJECTIVE

- 전투 액션 게임의 입력 반응성, 애니메이션 연속성, 서버 권한 상태 보정을 동시에 다룬다.
- 입력 계층의 직접 ChangeState() 호출을 제거하고 모든 상태 변경을 요청 기반 흐름으로 통합한다.
- StatePolicy를 중심으로 전이 허용, 거절, 즉시 전이, 서버 보정 명령 여부를 일관되게 관리한다.
- 복잡한 전투 상태 로직을 상태 실행, 렌더링 데이터, 표현 계층으로 분리하는 구조를 제안한다.

이론적 배경

BACKGROUND

- FSM은 게임 캐릭터 동작과 전이로 표현하는 대표적 구조이다.
- State, Strategy, Flyweight 관점에서 상태 실행 객체와 렌더링 데이터를 분리할 수 있다.
- 클라이언트 예측은 서버 응답 전에 로컬 결과를 먼저 반영해 입력 반응성을 높인다.
- 서버 보정은 로컬 예측과 충돌하지 않도록 현재 전투 상태를 고려해 적용되어야 한다.

문제 정의

PROBLEM DEFINITION

구분	직접 변경	요청 기반 정책
상태 변경	입력 코드가 목표 상태 결정	요청 전달
전이 규칙	여러 코드에 분산	StatePolicy 집중
서버 보정	로컬 상태 강제 덮어쓰기	정책 기반 환영
예외 처리	상황별 분기 증가	우선순위 기반 처리

구현 구조

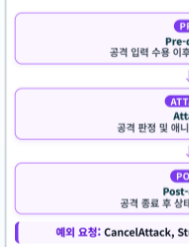
IMPLEMENTATION



입력과 서버 패킷은 직접 상태를 변경하지 않고 Request(State)를 수행한다. StatePolicy는 정책 상황과 요청 명령을 기준으로 허용, 거절, 즉시 전이 여부를 판단한다. 상태 객체는 StatePool에서 공유하고, 캐릭터별 현재 상태와 타이머, 공격 방향, 공격 직권은 FSMComponent가 보관한다.

공격 단계 분리

ATTACK PHASE SEPARATION



구조 검토

STRUCTURAL REVIEW

- 상태 객체 공유: 상태 객체는 개별 캐릭터 데이터를 보관하지 않는 Stateless 구조이며, StatePool이 상태 객체를 공유한다.
- 전이 규칙 분리: 캐릭터별 현재 상태, 타이머, 공격 방향, 공격 타입은 FSMComponent가 관리한다.
- 상태 실행: StateImplementation은 상태 진입, 갱신, 종료 시의 실제 동작을 담당한다.
- 정책 환영 절차: FSM 계층은 현재 상태와 요청 종류를 StatePolicy에 전달하고 허용·거절·즉시 전이를 환영받는다.
- 표현 계층 분리: FSM은 논리적 애니메이션 커맨드 선택하고, AnimationComponent와 AnimationLoader가 실제 리소스 연결과 재생을 담당한다.

로컬 예측과 서버 보정

LOCAL PREDICTION & SERVER CORRECTION

Key = f(D, T) = (D × BaseValue) + T

(D: 클라이언트 키 값, BaseValue: 공격 방향, T: 공격 타임)

- 로컬 클라이언트의 공격 요청은 FSM에 우선 반영하여 입력 반응성을 확보한다.
- 서버 패킷은 확인 패킷과 실제 보정이 필요한 패킷을 구분한다.
- 필요한 보정만 ForcedServerCorrection 요청으로 변환해 동일한 정책 경로에서 처리한다.
- 원격 클라이언트는 서버 권한 상태를 기준으로 상태와 표현 데이터를 갱신한다.

핵심 기여 및 결론

CONTRIBUTIONS & CONCLUSION

- 입력 처리, 상태 전이, 서버 보정의 책임을 요청 기반 경로로 통합하였다.
- StatePolicy 계층에 전이 판단을 집중시켜 전투 예외 처리와 서버 보정 기준을 일관화하였다.
- 공격 단계를 분리해 로컬 예측 중 서버 상태가 애니메이션을 즉시 중단시키는 상황을 제한하였다.
- 본 구조는 정량 성능보다 상태 전이 책임 분리와 정책 기반 흐름 정리에 초점을 뒀다.

한계 및 확장 방향

LIMITATIONS & EXTENSION

- 공격 후 이동 차단, 중복 공격 거절, 서버 보정 거절 규칙은 아직 StatePolicy의 초기 조건에 한계가 있다.
- 서버가 로컬 예측 공격을 거절했을 때 실제 공격 진행과 복귀 상태 결정 절차는 명시적으로 분리될 필요가 있다.
- 최소 가능 여부, 비동기 요청, 우선순위, 기본 다음 상태를 정책 데이터로 관리하면 클라이언트와 서버가 동일한 FSM 규칙을 공유할 수 있다.



(그림 3) 타이머 기반 상태 전이 정책 구조 예시

○ UI Architecture

개요

무엇을 구현했는가

전투 자세에서 플레이어 머리 위에 3방향 공격 UI를 띄우고, 마우스 이동으로 방향을 고른 뒤 좌클릭, 우클릭, 동시 입력으로 약공격, 강공격, 범위공격을 확정하는 구조를 구현했다.

왜 이러한 UI 구조가 필요했는가

전투 입력은 단순 버튼 입력이 아니라 방향 선택, 공격 종류 선택, FSM 상태 요청, 서버 패킷 전송, 애니메이션 키 선택이 한 번에 맞물려야 한다. 이 흐름을 분리하지 않으면 입력 지연, 중복 공격, 방향 불일치가 발생한다.